

NumPy (import numpy as np)

array initialization

```
np.array([2, 3, 4])           # direct initialization
np.empty(20, dtype=np.float32) # single precision array of size 20
np.zeros(200)                 # initialize 200 zeros
np.ones((3,3), dtype=np.int32) # 3 x 3 integer matrix with ones
np.eye(200)                   # ones on the diagonal
np.zeros_like(a)              # array with zeros and the shape of
a
np.linspace(0., 10., 100)     # 100 points from 0 to 10
np.arange(0, 100, 2)          # points from 0 to <100 with step 2
np.logspace(-5, 2, 100)       # 100 log-spaced from 1e-5 -> 1e2
np.copy(a)                    # copy array to new memory
```

indexing

```
a = np.arange(100)           # initialization with 0 - 99
a[:3] = 0                    # set the first three indices to zero
a[2:5] = 1                   # set indices 2-4 to 1
a[:-3] = 2                   # set all but last three elements to 2
a[start:stop:step]           # general form of indexing/slicing
a[None, :]                   # transform to column vector
a[[1, 1, 3, 8]]              # return array with values of the
indices
a = a.reshape(10, 10)        # transform to 10 x 10 matrix
a.T                           # return transposed view
b = np.transpose(a, (1, 0))  # transpose array to new axis order
a[a < 2]                      # values with elementwise condition
```

array properties and operations

```
a.shape                       # a tuple with the lengths of each axis
len(a)                        # length of axis 0
a.ndim                        # number of dimensions (axes)
a.sort(axis=1)                # sort array along axis
a.flatten()                   # collapse array to one dimension
a.conj()                      # return complex conjugate
```

```
a.astype(np.int16)           # cast to integer
a.tolist()                    # convert (possibly multidimensional) array
to list
np.argmax(a, axis=1)          # return index of maximum along a given axis
np.cumsum(a)                  # return cumulative sum
np.any(a)                     # True if any element is True
np.all(a)                     # True if all elements are True
np.argsort(a, axis=1)         # return sorted index array along axis
np.where(cond)                 # return indices where cond is True
np.where(cond, x, y)           # return elements from x or y depending on
cond
```

boolean arrays

```
a < 2                         # returns array with boolean values
(a < 2) & (b > 10)             # elementwise logical and
(a < 2) | (b > 10)            # elementwise logical or
```

elementwise operations and math functions

```
a * 5                         # multiplication with scalar
a + 5                         # addition with scalar
a + b                         # addition with array b
a / b                         # division with b (np.NaN for division by zero)
np.exp(a)                     # exponential (complex and real)
np.power(a, b)                 # a to the power b
np.sin(a)                     # sine
np.cos(a)                     # cosine
np.arctan2(a, b)               # arctan(a/b)
np.arcsin(a)                   # arcsin
np.radians(a)                  # degrees to radians
np.degrees(a)                  # radians to degrees
np.var(a)                      # variance of array
np.std(a, axis=1)              # standard deviation
```

reading/ writing files

```
np.save(fname/fobject, array)          # save as
numpy binary (.npy)
np.load(fname/fobject, mmap_mode='c')  # load .npy
file (memory mapped)
```

rounding

```
np.ceil(a)  # rounds to nearest upper int
np.floor(a) # rounds to nearest lower int
np.round(a) # rounds to nearest int
```

random variables

```
from np.random import normal, seed, rand, uniform, randint
normal(loc=0, scale=2, size=100) # 100 normal distributed
seed(23032)                      # resets the seed value
rand(200)                        # 200 random numbers in [0, 1)
uniform(1, 30, 200)              # 200 random numbers in [1, 30)
randint(1, 16, 300)              # 300 random integers in [1, 16)
```

Matplotlib (import matplotlib.pyplot as plt)

figures and axes

```
fig = plt.figure(figsize=(5, 2)) # initialize figure
fig.savefig('out.png')           # save png image
fig, axes = plt.subplots(5, 2, figsize=(5, 5)) # fig and 5 x 2
nparray of axes
ax = fig.add_subplot(3, 2, 2)    # add second subplot in a 3 x 2
grid
ax = plt.subplot2grid((2, 2), (0, 0), colspan=2) # multi
column/row axis
ax = fig.add_axes([left, bottom, width, height]) # add custom axis
```

figures and axes properties

```
fig.suptitle('title')           # big figure title
fig.subplots_adjust(bottom=0.1, right=0.8, top=0.9, wspace=0.2,
                      hspace=0.5) # adjust subplot positions
fig.tight_layout(pad=0.1, h_pad=0.5, w_pad=0.5,
                 rect=None)      # adjust subplots to fit into fig
ax.set_xlabel('xbla')           # set xlabel
ax.set_ylabel('ybla')           # set ylabel
ax.set_xlim(1, 2)               # sets x limits
ax.set_ylim(3, 4)               # sets y limits
ax.set_title('blabla')          # sets the axis title
ax.set(xlabel='bla')             # set multiple parameters at once
ax.legend(loc='upper center')    # activate legend
ax.grid(True, which='both')      # activate grid
bbox = ax.get_position()         # returns the axes bounding box
bbox.x0 + bbox.width             # bounding box parameters
```

plotting routines

```
ax.plot(x,y, '-o', c='red', lw=2, label='bla') # plots a line
ax.scatter(x,y, s=20, c=color)                 # scatter plot
ax.pcolormesh(xx, yy, zz, shading='gouraud')    # fast colormesh
ax.colormesh(xx, yy, zz, norm=norm)             # slower colormesh
ax.contour(xx, yy, zz, cmap='jet')              # contour lines
ax.contourf(xx, yy, zz, vmin=2, vmax=4)         # filled contours
n, bins, patch = ax.hist(x, 50)                # histogram
ax.imshow(matrix, origin='lower',
           extent=(x1, x2, y1, y2))            # show image
ax.specgram(y, FS=0.1, noverlap=128,
            scale='linear')                    # plot a
spectrogram
ax.text(x, y, string, fontsize=12, color='m')  # write text
```